

# API DOCUMENTATION

Chat Application — Backend API Reference  
Version 1.0 | May 2026 | Base URL: <https://api.chatapp.com/v1>

## API Overview

All APIs use **HTTPS** and return responses in **JSON** format. Authenticated endpoints require a Bearer token in the Authorization header: `Authorization: Bearer <token>`

| #  | API Group      | Method | Endpoint                | Auth Required |
|----|----------------|--------|-------------------------|---------------|
| 1  | Authentication | POST   | /auth/register          | No            |
| 2  | Authentication | POST   | /auth/login             | No            |
| 3  | Authentication | POST   | /auth/logout            | Yes           |
| 4  | User / Profile | GET    | /user/:id               | Yes           |
| 5  | User / Profile | PUT    | /user/:id               | Yes           |
| 6  | Chat List      | GET    | /chats                  | Yes           |
| 7  | Messages       | GET    | /chats/:chatId/messages | Yes           |
| 8  | Messages       | POST   | /chats/:chatId/messages | Yes           |
| 9  | WebSocket      | WS     | /ws/chat                | Yes           |
| 10 | File Upload    | POST   | /upload                 | Yes           |
| 11 | Search         | GET    | /chats/search?q=        | Yes           |

## 1. Authentication API

Handles user registration, login, and session management. No authentication token is required for register and login endpoints.

**POST**     **/auth/register**

Creates a new user account. Validates that the email is not already registered and that passwords match.

**Request Body (JSON):**

```
{ "name": "John Doe", "email": "john@example.com", "password": ".....",  
  "confirmPassword": "....." }
```

**Response (JSON):**

```
{ "success": true, "message": "Account created successfully" }
```

■ Returns 409 Conflict if the email is already registered.

**POST****/auth/login**

Validates user credentials and returns a session token used for all subsequent authenticated requests.

**Request Body (JSON):**

```
{ "email": "john@example.com", "password": "....." }
```

**Response (JSON):**

```
{ "success": true, "token": "eyJhbGciOiJIUzI1NiIs... ", "user": { "id": "u123", "name":  
  "John Doe", "email": "john@example.com" } }
```

■ Returns 401 Unauthorized if credentials are invalid. Token expires after 30 minutes of inactivity.

**POST****/auth/logout**

Terminates the current user session and invalidates the Bearer token.

**Response (JSON):**

```
{ "success": true, "message": "Logged out successfully" }
```

■ Requires Authorization: Bearer header.

## 2. User / Profile API

Allows users to fetch and update their personal profile information including name and profile photo.

**GET****/user/:id**

Fetches the profile details of a user by their unique ID.

**Parameters:**

- **:id** — The unique identifier of the user (e.g., u123)

**Response (JSON):**

```
{ "id": "u123", "name": "John Doe", "email": "john@example.com", "profilePhoto":  
  "https://cdn.chatapp.com/photos/u123.jpg" }
```

■ Returns 404 Not Found if the user does not exist.

**PUT****/user/:id**

Updates the name and/or profile photo of the authenticated user. Only the owner can update their own profile.

**Parameters:**

- **:id** — The unique identifier of the user to update

**Request Body (JSON):**

```
{ "name": "John Updated", "profilePhoto": "" }
```

**Response (JSON):**

```
{ "success": true, "message": "Profile updated successfully", "user": { "id": "u123", "name": "John Updated" } }
```

■ Returns 403 Forbidden if the token does not match the target user ID.

### 3. Chat List API

Returns all conversations belonging to the authenticated user, including metadata needed to render the Chat List Screen.

**GET**    **/chats**

Fetches all conversations for the logged-in user, sorted by most recent message timestamp (descending).

**Response (JSON):**

```
{ "chats": [ { "chatId": "c001", "contactName": "Alice", "profilePhoto": "https://...", "lastMessage": "Hey, how are you?", "timestamp": "2026-05-31T10:30:00Z", "unreadCount": 3 } ] }
```

■ Pagination supported via query params: `?page=1&limit=20`

### 4. Messages API

Handles loading message history and sending new messages within a specific conversation.

**GET**    **/chats/:chatId/messages**

Retrieves the full message history for a specific conversation, ordered chronologically.

**Parameters:**

- **:chatId** — The unique identifier of the conversation

**Response (JSON):**

```
{ "messages": [ { "msgId": "m001", "senderId": "u123", "text": "Hello!", "timestamp": "2026-05-31T10:00:00Z", "type": "text" } ] }
```

■ Supports cursor-based pagination: `?before=&limit=50`. Maximum 10,000 messages per conversation.

**POST**    **/chats/:chatId/messages**

Sends a new message in the specified conversation. Message is also broadcast in real time via WebSocket.

**Parameters:**

- **:chatId** — The unique identifier of the conversation

**Request Body (JSON):**

```
{ "text": "Hello, how are you?", "type": "text" // or "image", "file" }
```

**Response (JSON):**

```
{ "success": true, "message": { "msgId": "m042", "senderId": "u123", "text": "Hello, how are you?", "timestamp": "2026-05-31T10:35:00Z" } }
```

■ For media messages, first upload the file via `POST /upload` and include the returned URL in the text field.

## 5. WebSocket API (Real-Time)

Provides a persistent bidirectional connection for real-time features including instant message delivery, online/offline status, and unread count updates.

**WebSocket**  
et `/ws/chat`

Establishes a persistent WebSocket connection authenticated via token query parameter.

**Parameters:**

- **token** — Bearer token passed as query param: `/ws/chat?token=`

**Request Body (JSON):**

```
// Client sends: { "type": "message", "chatId": "c001", "text": "Hey!" } // Client sends
typing indicator: { "type": "typing", "chatId": "c001" }
```

**Response (JSON):**

```
// Server pushes new message: { "type": "new_message", "chatId": "c001", "message": {
"msgId": "m043", "text": "Hey!", ... } } // Server pushes status update: { "type":
"status", "userId": "u456", "online": true }
```

■ Connection auto-reconnects on drop. Ping/pong heartbeat every 30 seconds to keep connection alive.

## 6. File / Attachment API

Handles uploading of images and file attachments. Returns a CDN URL to be embedded in a message.

**POST** `/upload`

Uploads a file or image to the CDN. The returned URL is then included in a message sent via POST `/chats/:chatId/messages`.

**Request Body (JSON):**

```
// multipart/form-data file: chatId: "c001"
```

**Response (JSON):**

```
{ "success": true, "url": "https://cdn.chatapp.com/uploads/file_abc123.jpg", "type":
"image", "sizeKB": 248 }
```

■ Maximum file size: 10 MB. Supported types: jpg, png, gif, pdf, mp4, mp3, zip.

## 7. Search API

Enables users to search through their conversations by contact name or message content.

**GET** `/chats/search?q=keyword`

Searches across all conversations of the authenticated user for matching contact names or message text.

**Parameters:**

- **q** — Search keyword (minimum 2 characters)
- **limit** — Optional. Max results to return (default: 20)

**Response (JSON):**

```
{ "results": [ { "chatId": "c001", "contactName": "Alice", "matchedMessage": "Hey, how are you?", "timestamp": "2026-05-31T10:30:00Z" } ] }
```

■ Returns an empty results array (not 404) when no matches are found.

## 8. Common HTTP Status Codes

| Status Code      | Meaning          | When It Occurs                        |
|------------------|------------------|---------------------------------------|
| 200 OK           | Success          | Request processed successfully        |
| 201 Created      | Resource Created | New user or message created           |
| 400 Bad Request  | Invalid Input    | Missing fields or validation failure  |
| 401 Unauthorized | Auth Failed      | Invalid or missing Bearer token       |
| 403 Forbidden    | Access Denied    | Token valid but user lacks permission |
| 404 Not Found    | Resource Missing | User, chat, or message ID not found   |
| 409 Conflict     | Duplicate Entry  | Email already registered              |
| 413 Too Large    | File Too Big     | Upload exceeds 10 MB limit            |
| 500 Server Error | Internal Error   | Unexpected server-side failure        |